UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE MATEMÁTICA E ESTATÍSTICA
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

Rafael Vieira de Carvalho

Divisão de nós em uma max-tree

Atualização dinâmica de max-trees sob transformações anti-extensivas locais

Orientador: Prof. Dr. Ronaldo F. Hashimoto
Coorientador: Prof. Dr. Wonder A. L. Alves

São Paulo
2026

1 Resumo

Este projeto investiga a atualização dinâmica de *component trees*, especialmente a *max tree*, para lidar com alterações locais em imagens sem reconstruir toda a estrutura; a morfologia matemática fornece o alicerce teórico para essa questão, baseada em conectividade, limiares e componentes conexas. O problema central é que mudanças no nível de cinza podem fragmentar, fundir ou deslocar nós da árvore, tornando a representação anterior inconsistente e exigindo reprocessamento completo. Assim, o estudo propõe formalizar e desenvolver a operação de *split*, que divide um nó em múltiplos filhos quando uma componente se fragmenta, buscando eficiência, correção e atualizações locais.

Os objetivos incluem responder como a árvore deve ser modificada, quais estruturas de dados permitem manter o custo baixo, como recalcular atributos incrementalmente e qual ganho prático se obtém comparada a reconstrução completa.

2 Introdução e Justificativas

A morfologia matemática é uma subárea do processamento de imagens e da visão computacional, cujo objetivo consiste em estudar e desenvolver transformações aplicadas a imagens, tendo como principal arcabouço teórico a teoria dos conjuntos [8]. Conforme sugere o próprio termo *morfologia*, essa área busca analisar e extrair informações relacionadas às características geométricas dos objetos presentes nas imagens, como forma, tamanho e estrutura.

Nesse contexto, o uso de estruturas de dados para representar e aplicar transformações em imagens é essencial. Exemplos de estruturas incluem quad-trees [15, 3], *binary partition tree* [11] e *watershed tree* [7].

A *component tree* [12] é uma das estruturas utilizadas para representar imagens. A ideia fundamental por trás dessa estrutura consiste em representar uma imagem a partir da ordenação de seu conjunto de pixels e de suas componentes conexas. Para isso, é necessário definir o conceito de componente conexa em uma imagem [10, 9] e, de forma mais geral, estabelecer como uma imagem pode ser representada por meio de um grafo [8].

A representação de uma imagem como grafo é realizada de forma direta: os vértices correspondem aos pixels da imagem, enquanto as arestas são definidas conforme um determinado modelo de conectividade [10].

Conforme observado anteriormente, uma das premissas para a construção da *component*

tree é a ordenação do conjunto de pixels. Existem duas formas de ordenar os pixels: de maneira ascendente ou descendente. Essas formas dão origem a duas estruturas de dados duais: a *max tree*, cuja ordenação é ascendente, isto é, a relação de ordenação dos pixels é dada pelo operador $\leq$ entre os níveis de cinza; e a *min tree*, cuja ordem é descendente, isto é, a relação de ordenação dos pixels é dada pelo operador $\geq$ entre os níveis de cinza.

A *component tree* foi originalmente desenvolvida para realizar operações conexas com o propósito de resolver tarefas de filtragem e segmentação [13, 14]. Nesse contexto, a operação mais comum é o corte, que pode consistir na remoção de alguns nós com base na otimização de algum critério ou na eliminação de nós cujos atributos satisfaçam determinada regra.

Tal estrutura é amplamente utilizada devido ao seu baixo custo computacional de construção. Segundo [6], uma imagem com n pixels pode ter sua *component tree* construída em tempo quasi-linear. Além dessa característica, a *component tree* permite extrair informações morfológicas relevantes das imagens. Por esse motivo, essa estrutura tem sido empregada em abordagens de aprendizado profundo para a extração de características, conforme apresentado em [1, 2, 5].

Nesse contexto, uma dificuldade encontrada na utilização de *component trees* é a atualização dessa estrutura quando a imagem sofre alguma alteração. Em outras palavras, quando a imagem é modificada, não existe um algoritmo genérico que atualize a *component tree* de forma a representá-la corretamente após a transformação. Em [4] apenas um dos operadores para atualizar a *component tree* é proposto. Diante disso, torna-se necessário reconstruir a árvore completa a cada mudança na imagem.

Em [4], os autores propõem a operação de *merge* (mescla) de nós para resolver parcialmente o problema de atualização. Como também observado em [4], o problema dual, *split*/divisão, também precisa ser resolvido para que seja possível atualizar completamente a árvore.

Sob esse contexto, este projeto de pesquisa pretende contribuir para essa lacuna na literatura. Para simplificar, assim como em [4], o estudo será restrito à *max tree*.

Este documento está organizado da seguinte forma. Na Seção 3, serão introduzidas algumas das notações, propriedades e definições apresentadas em [4]. Em posse desse conhecimento teórico, na Seção 4 será apresentada a pergunta de pesquisa de maneira formal, assim como as perguntas auxiliares que a orientam. Na Seção 5, serão descritos os métodos a serem empregados para alcançar os objetivos propostos, bem como os procedimentos adotados

para garantir a reprodutibilidade da pesquisa. Na Seção 6, será apresentado o planejamento das atividades e a priorização das etapas a serem executadas, de modo a assegurar o desenvolvimento adequado da pesquisa. Por fim, na Seção 7, serão discutidos os possíveis riscos associados à pesquisa e propostas estratégias de mitigação e alternativas para os casos em que o planejamento originalmente estabelecido não produza os resultados esperados.

3 Notações e teoria

3.1 Conjuntos

Seja A um conjunto. O conjunto potência de A, ou conjunto das partes, é denotado por 2^A. Seja S um conjunto de subconjuntos de 2^A ($S \subseteq 2^A$); escrevemos:

$$\bigcup S = \bigcup_{X \in S} X$$

3.2 Relações

Seja $a, b \in A$ e seja $\ltimes$ uma relação binária em A, ou seja, $\ltimes \subseteq A \times A$. Usamos a notação $a \ltimes b$ para expressar o elemento $(a, b) \in \ltimes$.

Denotamos por $\rtimes$ a relação inversa associada a $\ltimes$, isto é, $(a \rtimes b) \iff (b \ltimes a)$.

Denotamos o conjunto de elementos tais que a relação $a \ltimes b$ é verdadeira por:

$$\ltimes(a) = \{b \in A : a \ltimes b\}$$

3.3 Funções

Seja B um conjunto. Uma função (ou uma aplicação) $\mathcal{F}$ de A para B é uma parte de $A \times B$ tal que, para qualquer $a \in A$, existe exatamente um $b \in B$ tal que $(a, b) \in \mathcal{F}$. Escrevemos $b = \mathcal{F}(a)$ para expressar tal par ordenado. Denotamos por B^A o conjunto de todas as funções que levam A para B.

Seja $\widetilde{A} \subseteq A$. Define-se o conjunto $\mathcal{F}(\widetilde{A}) = \{\mathcal{F}(a) : a \in \widetilde{A}\}$. Quando $\mathcal{F}(\widetilde{A}) = \{b\}$ escrevemos $\mathcal{F}(\widetilde{A}) = b$ por abuso de notação. Seja $\widetilde{B} \subseteq B$, escrevemos $\mathcal{F}^{-1}(\widetilde{B}) = \{a \in A : \mathcal{F}(a) \in \widetilde{B}\}$. Quando $\widetilde{B} = \{b\}$ escrevemos $\mathcal{F}^{-1}(b)$ ao invés de $\mathcal{F}^{-1}(\{b\})$ por abuso de notação.

3.4 Ordem

Seja $\preccurlyeq$ uma relação binária. Dizemos que $\preccurlyeq$ é uma ordem em A, mais precisamente que $(A, \preccurlyeq)$ é um conjunto ordenado, se a relação é reflexiva, transitiva e antissimétrica. Dizemos que $\preccurlyeq$ é totalmente ordenado se, além de ser ordenado, também possui a condição de comparabilidade (ou totalidade). Dizemos que $\preccurlyeq$ é uma ordem parcial em A, ou *poset*, se não possui a condição de comparabilidade.

Suponha que $(A, \preccurlyeq)$ é um conjunto ordenado. Seja $\widetilde{A} \subseteq A$. Define-se o seguinte:

- O supremo de $(\widetilde{A}, \preccurlyeq)$ é denotado por $\bigvee^{\preccurlyeq} \widetilde{A}$.
- O ínfimo de $(\widetilde{A}, \preccurlyeq)$ é denotado por $\bigwedge^{\preccurlyeq} \widetilde{A}$.
- O conjunto dos elementos maximais de $(A, \preccurlyeq)$ é denotado por $\nabla^{\preccurlyeq} \widetilde{A}$.
- O conjunto dos elementos minimais de $(A, \preccurlyeq)$ é denotado por $\Delta^{\preccurlyeq} \widetilde{A}$.

O diagrama de Hasse do conjunto ordenado $(A, \preccurlyeq)$ é denotado por $(A, \lhd)$ e corresponde à remoção de redundâncias das propriedades reflexivas e transitivas. Por abuso de notação, o diagrama de Hasse no conjunto ordenado $(\widetilde{A}, \preccurlyeq)$ também é denotado por $(\widetilde{A}, \lhd)$.

Quando $(A, \preccurlyeq)$ é um conjunto totalmente ordenado e $a \preccurlyeq b$ com $a, b \in A$, denotamos o intervalo de elementos entre a e b por

$$[\![a, b]\!] = \{c \in A : a \preccurlyeq c \preccurlyeq b\}$$

3.5 Árvores

Seja A um conjunto não vazio e $\preccurlyeq$ uma ordem em A. O conjunto ordenado $(A, \preccurlyeq)$ é considerado de ordem hierárquica superior se as seguintes condições são satisfeitas:

1. $(A, \preccurlyeq)$ admite máximo $\bigvee^{\preccurlyeq} \widetilde{A}$.
2. $\forall a \in A$, o conjunto $(\preccurlyeq (a), \preccurlyeq)$ é totalmente ordenado.

Sob essas condições, podemos dizer que, se $(A, \preccurlyeq)$ é uma ordem hierárquica superior, o diagrama de Hasse $(A, \lhd)$ de tal conjunto ordenado é uma árvore em que:

1. A raiz é $\bigvee^{\preccurlyeq} A$.

2. O conjunto de folhas é $\Delta^{\preccurlyeq} A$.

3. O conjunto de arestas é $\lhd$.

A relação $\lhd$ pode ser vista como uma função $\lhd : A \to A$ e, portanto, podemos escrever $\lhd(a) = b$ ao invés de $a \lhd b$.

Seja $\emptyset \subset \widetilde{A} \subseteq A$. Uma subárvore de $(A, \lhd)$ é definida por:

$$\overset{\triangle}{A} = \bigcup_{a \in \widetilde{A}} [\![a, \bigvee^{\preccurlyeq} \widetilde{A}]\!]$$

Ou seja, a árvore $(\overset{\triangle}{A}, \lhd)$ é uma subárvore de $(A, \lhd)$. Uma propriedade importante é: se $\overset{\triangle}{A} \neq A$ então:

- $(A \setminus \overset{\triangle}{A}, \preccurlyeq)$ continua sendo um conjunto ordenado superior.
- $(A \setminus \overset{\triangle}{A}, \lhd)$ continua sendo uma árvore.

3.6 Árvore de componentes

Com as definições de conjuntos, relações, funções, ordem e árvores agora podemos definir o que é uma imagem e o que é a árvore de componentes dessa imagem.

Definição 1 (relação de adjacência)**.** Seja $\mathbb{P}$ um conjunto finito. Uma relação binária $\frown \subseteq \mathbb{P} \times \mathbb{P}$ é dita de adjacência se for irreflexiva e simétrica.

Definição 2 (relação de conectividade)**.** Uma relação de adjacência $\frown$ induz uma relação de conectividade $\sim$ em $\mathbb{P}$. Dois elementos $x, y \in \mathbb{P}$ são ditos conexos ou conectados ($x \sim y$) se e somente se $x \frown y$ ou se existe uma sequência finita com $k \geq 1$ formada por elementos $x_0, x_1, \ldots, x_k \in \mathbb{P}$ tal que:

1. $x_0 = x$.

2. $x_k = y$.

3. $x_i \frown x_{i+1}$ para todo $i \in \{0, 1, \ldots, k-1\}$.

Definição 3. Um conjunto $X \subseteq \mathbb{P}$ é dito ser uma componente conexa se for maximal relativamente à propriedade de conexidade.

Para qualquer $X \subseteq \mathbb{P}$, o conjunto de componentes conexas de X é denotado por $\Pi(X)$ (com $\Pi(\emptyset) = \emptyset$). Note que, quando $X \neq \emptyset$, o conjunto $\Pi(X)$ é uma partição de X. Quando $X \neq \emptyset$, usamos $\mathscr{C}(X)$ para indicar que X é conexo, isto é, que possui apenas uma componente conexa ($\Pi(X) = \{X\}$).

Definição 4. Seja $\mathbb{V} \subseteq \mathbb{Z}$ um conjunto finito com a relação de ordem $\leq$. Seja $\perp$ e $\top$ os menores e maiores elementos de $\mathbb{V}$, respectivamente. Uma imagem em níveis de cinza I é uma função $I : \mathbb{P} \to \mathbb{V}$, onde o domínio $\mathbb{P}$ (comumente chamado de *pixels*) possui relação de adjacência e o contradomínio corresponde aos níveis de cinza da imagem.

Seja $v \in \mathbb{V}$. A função de limiar para um valor v é definida como

$$\left| \begin{array}{ll} \lambda_v : & \mathbb{V}^{\mathbb{P}} \longrightarrow 2^{\mathbb{P}} \\ & \mathcal{F} \longrightarrow \{x \in \mathbb{P} : v \leq \mathcal{F}(x)\} \end{array} \right. \tag{1}$$

O limiar para um nível de cinza v para uma imagem I é definido como

$$\Lambda_v = \lambda_v(I)$$

O conjunto de componentes conexas é

$$\Theta_v = \Pi(\Lambda_v)$$

Seja o intervalo $[\![\alpha, \beta]\!] \subseteq \mathbb{V}$, o conjunto de todas as componentes conexas para os valores de v é definido por

$$\Theta_{[\![\alpha,\beta]\!]} = \bigcup_{v \in [\![\alpha,\beta]\!]} \Theta_v$$

Portanto, o conjunto de todas as componentes conexas para todos os limiares é denotado por

$$\Theta = \Theta_{[\![\perp,\top]\!]}$$

A partir da definição de Θ agora podemos introduzir o conceito de árvore de componentes. Considere a relação binária parcial $\subseteq$ em Θ, ou seja, considere o conjunto de ordem parcial $(\Theta, \subseteq)$. Note que tal conjunto é considerado de ordem hierárquica superior devido ao conceito

apresentado em 3.5, pois:

1. O máximo de tal conjunto corresponde a $\mathbb{P} = \bigvee^{\subseteq} \Theta$.

2. Para qualquer componente $X_v \in \Theta_v$ ($v \in [\![\bot + 1, \top]\!]$) existe exatamente uma componente $X_{v-1} \in \Theta_{v-1}$ tal que $X_v \subseteq X_{v-1}$. Portanto, $(\subseteq (X), \subseteq)$ é totalmente ordenado.

Conforme apresentado em 3.5, o diagrama de Hasse de $(\Theta, \subseteq)$ é uma árvore. Como essa árvore construída armazena a informação das componentes conexas, a ela denominamos de árvore de componentes; mais especificamente, a árvore construída é a *max tree*.

Definição 5. A *max tree* $\mathscr{T}$ é o diagrama de Hasse $(\Theta, \lhd)$ do conjunto de ordem parcial $(\Theta, \subseteq)$. Os elementos de Θ são os nós da árvore. A raiz é $\bigvee^{\subseteq} \Theta$. As folhas são os elementos de $\Delta^{\subseteq}\Theta$. As arestas são os elementos de $\lhd$.

A título de curiosidade, a *max tree* possui uma árvore dual chamada *min tree*; para sua construção basta substituir o operador $\leq$ por $\geq$ na equação (1).

É fácil notar que essa definição de *max tree* não é computacionalmente eficiente em termos de memória. Em [4] é adotada uma versão mais eficiente. Como o propósito deste projeto de pesquisa é apenas contextualizar o problema, tal assunto não será abordado aqui.

3.7 Merge

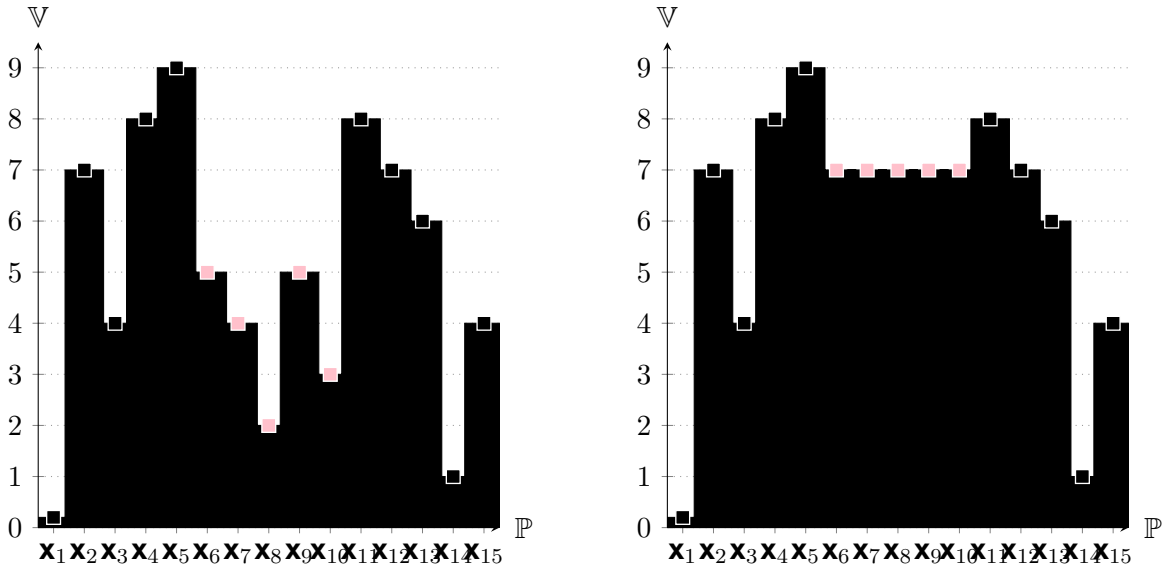
Conforme explicado em 2, a árvore de componentes é amplamente utilizada para diversas operações. No entanto, quando a imagem é alterada faz-se necessária a reconstrução total da árvore. Nesse sentido, [4] propõe a operação de mescla (*merge*) entre os nós da árvore. A seguir formular-se-á o problema da mescla com o intuito de fornecer o contexto para o objetivo desta pesquisa.

Pergunta 1: sejam $I^- : \mathbb{P} \to \mathbb{V}$ uma imagem antes de sofrer alguma transformação, $\mathscr{T}^-_{\mathsf{max}}$ a *max tree* associada a I^-, $C \subset \mathbb{P}$ um conjunto conexo, $a = \bigwedge^{\leq} I^-(C)$ e $b \in \mathbb{V}$ tal que $b > \bigvee^{\leq} I^-(C)$. A imagem I^+ é definida como:

$$
\left|\begin{array}{lll}
I^+ : & \mathbb{P} & \to \quad \mathbb{V} \\
& x \in \mathbb{P} \setminus C & \to \quad I^-(x) \\
& x \in C & \to \quad b
\end{array}\right.
$$

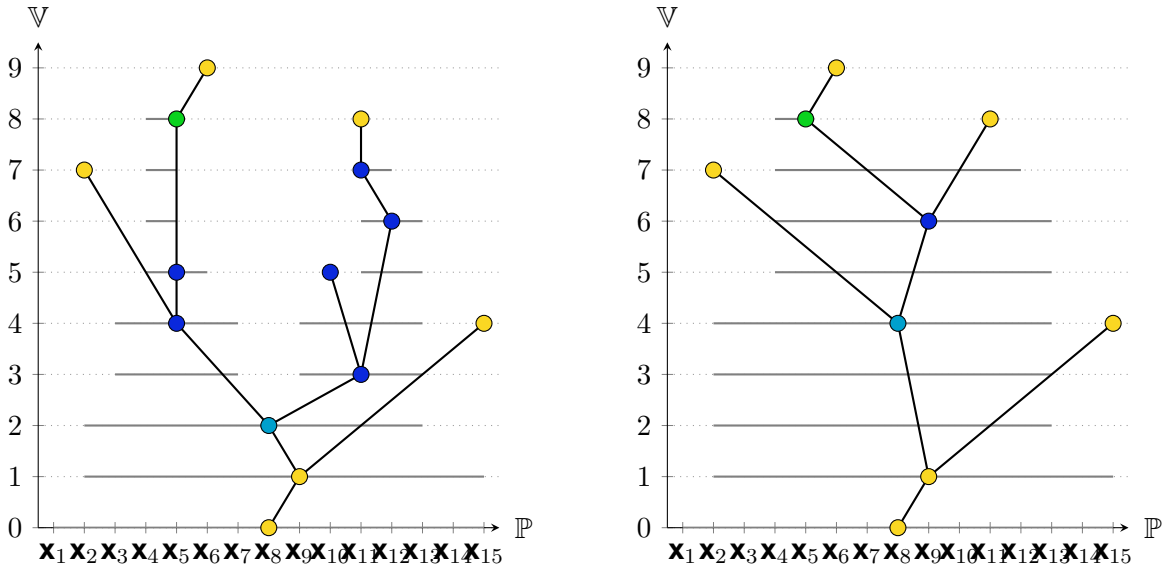
Seja $\mathscr{T}^+_{\max}$ a *max tree* associada a I^+. Sob essas condições, como devemos modificar $\mathscr{T}^-_{\max}$ para obter $\mathscr{T}^+_{\max}$?

Pergunta 2: sejam $\mathscr{T}^-_{\min}$ a *min tree* de I^- e $\overset{\triangle}{\mathscr{T}}{}^-_{\min}$ uma subárvore dessa *min tree*. Seja $\mathscr{T}^+_{\min}$ a *min tree* obtida ao remover a subárvore $\overset{\triangle}{\mathscr{T}}{}^-_{\min}$ de $\mathscr{T}^-_{\min}$. Seja I^+ a imagem reconstruída a partir de $\mathscr{T}^+_{\min}$. Como modificar $\mathscr{T}^-_{\max}$ para obter $\mathscr{T}^+_{\max}$ sob essas condições?



(a) I^- (b) I^+

Figura 1: As imagens acima ilustram o cenário da **pergunta 1**. Exemplo retirado de [4]



(a) $\mathscr{T}^-_{\max}$ (b) $\mathscr{T}^+_{\max}$

Figura 2: *Max tree* das imagens I^- e I^+ respectivamente. Exemplo retirado de [4]

4 Objetivos

Objetiva-se resolver o problema dual proposto na **Pergunta 1**, isto é, sejam $I^- : \mathbb{P} \to \mathbb{V}$ uma imagem antes de sofrer alguma transformação, $\mathscr{T}^-_{\mathsf{max}}$ a *max tree* associada a I^-, $C \subset \mathbb{P}$ um conjunto conexo, $a = \bigwedge^{\leq} I^-(C)$ e $b \in \mathbb{V}$ tal que $b < \bigwedge^{\leq} I^-(C)$. A imagem I^+ é definida como:

$$\left| \begin{array}{lll} I^+ : & \mathbb{P} & \to \quad \mathbb{V} \\ & x \in \mathbb{P} \setminus C & \to \quad I^-(x) \\ & x \in C & \to \quad b \end{array} \right.$$

Pergunta de pesquisa: Seja $\mathscr{T}^+_{\mathsf{max}}$ a *max tree* associada a I^+. Sob essas condições, como deve ser modificada $\mathscr{T}^-_{\mathsf{max}}$ para obter $\mathscr{T}^+_{\mathsf{max}}$?

Resolver esse problema dual implica a elaboração de um algoritmo eficiente de divisão de nós em uma *max tree*. Para tanto, tornam-se necessárias questões auxiliares sobre modelagem teórica e topológica, complexidade algorítmica e estruturas de dados, manutenção dinâmica de atributos, generalização e validação.

Pergunta sobre modelagem teórica: Como definir formalmente a reestruturação das componentes conexas e a redistribuição de arestas quando a redução de níveis de cinza em um conjunto de pixels provoca a fragmentação de um nó da *max tree* em múltiplos nós-filhos?

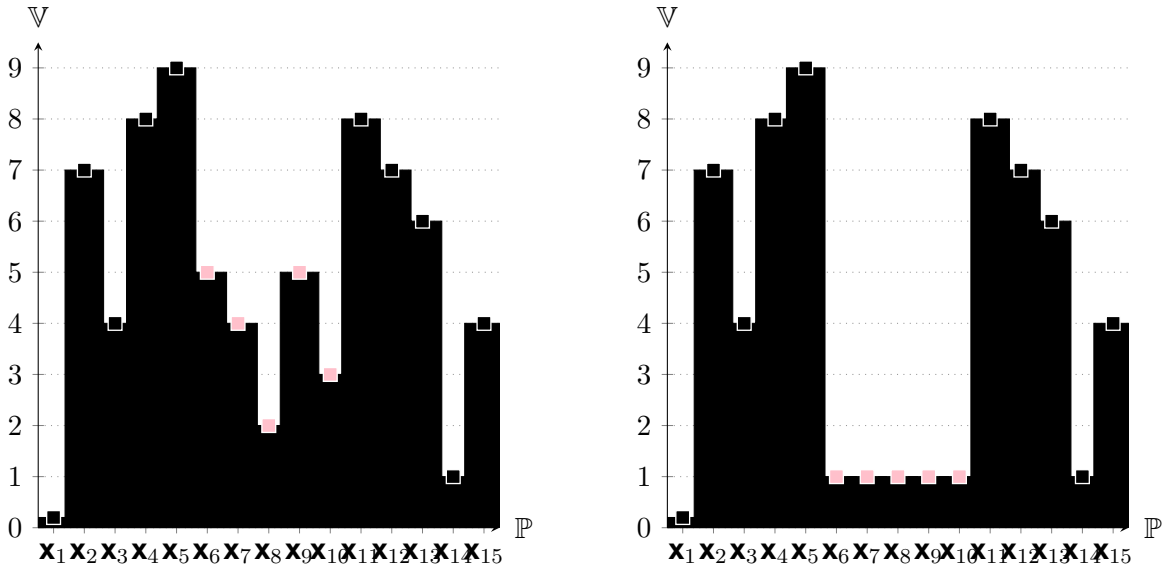
Pergunta sobre complexidade algorítmica e estruturas de dados: Quais estruturas de dados e rotinas de rastreamento local permitem limitar a complexidade da operação de *split*, evitando que ela degrade para o custo de uma reconstrução quasi-linear?

Pergunta sobre manutenção dinâmica de atributos: De que maneira os atributos dos nós (como área, momentos e caixas delimitadoras) podem ser recalculados de forma local e incremental quando a massa de pixels de um nó é particionada entre novas ramificações da árvore?

Pergunta sobre generalização: Como combinar as operações de *merge* (já estudadas) e de *split* em um modelo unificado que suporte alterações arbitrárias e simultâneas de níveis de cinza em pontos isolados da imagem? Existem outros operadores necessários para essa generalização?

Pergunta sobre validação: Qual é o ganho de desempenho computacional obtido com o operador de *split* em filtros conectados complexos em imagens de grande escala, em comparação com os métodos tradicionais de reconstrução?

Como o foco da pesquisa será o operador *split*, a pergunta de generalização será respondida somente se houver tempo sobrando na pesquisa.



(a) I^- (b) I^+

Figura 3: As imagens acima ilustram o cenário da **pergunta de pesquisa**.

5 Metodologia

Como mencionado na seção anterior, não basta elaborar o algoritmo que responde à pergunta de pesquisa. É necessário garantir que o algoritmo proposto funcione em todas as instâncias do problema e apresente desempenho superior ao da reconstrução completa da árvore de componentes. Nesta seção propõem-se os passos necessários para assegurar que o algoritmo desenvolvido seja eficiente e correto em todas as instâncias do problema.

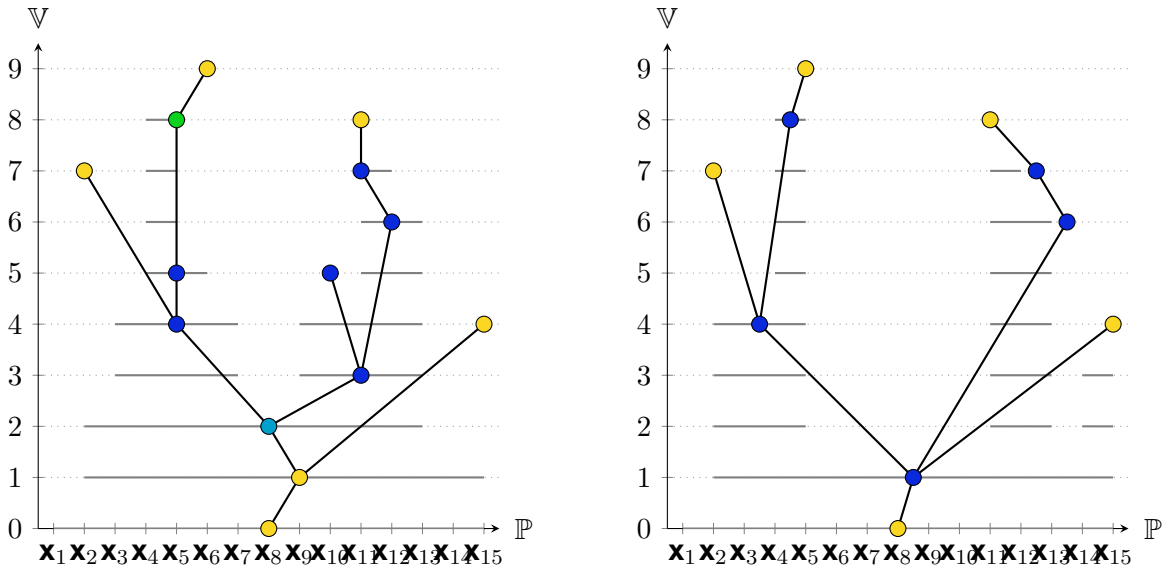


Figura 4: *Max tree* das imagens I^- e I^+ respectivamente. Árvores referente a figura 3

Em primeiro lugar, será pesquisado e construído um arcabouço teórico. Esse arcabouço fornecerá o rigor matemático necessário para demonstrar a correção e as propriedades de complexidade do algoritmo proposto. Além da construção teórica, inclui-se uma etapa dedicada a demonstrações formais do arcabouço.

Em segundo lugar, com o alicerce matemático estabelecido, será elaborada a versão algorítmica que realiza a fragmentação (”split”) de nós em uma *max tree*. Caso a construção de um algoritmo eficiente se prove impossível, o arcabouço permitirá justificar formalmente essa impossibilidade. Quando o algoritmo for proposto, pretende-se demonstrar sua corretude e analisar sua complexidade em termos de tempo e espaço. Nesta etapa será apresentado um pseudocódigo.

Em terceiro lugar, o algoritmo será implementado primeiramente em python para demonstrar a viabilidade prática da solução. Em seguida será realizado uma conversão de python para C++ para a etapa de testes, a qual sua importância será explicada a seguir. Além das demonstrações conceituais, para assegurar que a implementação está correta irão ser implementados testes unitários automatizados para a operação de split. O óraculo para estes testes é simples, basta comparar os nós da árvore que sofreu o *split* com a árvore que será reconstruída.

Em quarto lugar, utilizar-se-á a implementação em C++ para a coleta de dados pois o executável gerado por compiladores dessa linguagem costumam empiricamente possuir perfomance melhor se comparado a línguagens interpretadas como python. O dataset que será utilizado

será o *Occluded RoadText Challange* para podermos fazer uma comparação direta com os resultados de [4]. Algumas métricas que serão coletadas são: número de subárvores geradas, número de nós gerados, tempo de execução etc. Os fatores experimentais que irão ser controlados para os testes nesse dataset são: resolução e escala da imagem, tamanho da região afetada pela transformação(C), amplitude do intervalo de valores ($[a, b]$)

Por fim, relizar-se-á um estudo de caso para demonstrar, na prática, os contextos nos quais o operador de *split* pode trazer benefício em relação às abordagens tradicionais.

6 Plano de trabalho e cronograma

O projeto está em fase inicial. Atualmente, o ambiente de execução do algoritmo está sendo configurado e as rotinas e automações para suporte ao desenvolvimento futuro estão em implementação.

Atividade	**Jan**	**Fev**	**Mar**	**Abr**	**Mai**	**Jun**	**Jul**	**Ago**	**Set**	**Out**	**Nov**	**Dez**
Revisão bibliográfica	■	■				■			■			
Formalização matemática	■	■	■	■	■	■	■	■				
Algoritmo de split		■	■	■	■	■	■	■				
Implementação		■	■	■	■	■	■	■				
Testes e estudo de caso									■	■	■	■
Escrita da tese e documentação	■	■	■	■	■	■	■	■	■	■	■	■
Qualificação	■											
Defesa e depósito												■

Tabela 1: Plano de trabalho para 2027

7 Mitigação de Riscos

O projeto envolve desafios teóricos, algorítmicos e experimentais, de modo que é necessário antecipar possíveis dificuldades e definir estratégias de resposta. A seguir, são apresentados os principais riscos e as contingências previstas.

7.0.1 Formalização teórica da operação de split

Um risco central é a dificuldade de formalizar a divisão de um nó da *max tree* em múltiplos nós-filhos, especialmente no que tange à redistribuição de componentes conexas e de arestas após uma alteração local na imagem. Essa dificuldade pode comprometer tanto a correção do modelo quanto a clareza da demonstração matemática.

Como contingência, a formalização será conduzida em etapas progressivas: primeiro, com transformações simples e casos particulares; em seguida, com generalização para situações mais complexas. A construção de contraexemplos e casos limite será usada para validar a consistência do arcabouço teórico.

7.0.2 Eficiência e corretude do algoritmo

O operador de *split* pode não atingir o nível de eficiência desejado, ou ainda pode exigir estruturas de dados e procedimentos de manutenção mais complexos do que o inicialmente previsto. Nesse caso, o algoritmo pode perder vantagem em relação à reconstrução completa da árvore.

Como contingência, será adotada uma estratégia incremental de desenvolvimento, com análise de complexidade em cada etapa, além de manutenção de invariantes estruturais. A validação será feita tanto por demonstrações formais quanto por comparação experimental com a reconstrução completa.

7.0.3 Atualização de atributos dos nós

A recalculação de atributos como área, momentos e caixas delimitadoras pode ser mais custosa do que o esperado, sobretudo quando a fragmentação de um nó gera múltiplas ramificações. Esse risco pode afetar a viabilidade prática do método.

Como contingência, serão identificados atributos que podem ser atualizados localmente e incrementalmente, e serão propostas estruturas auxiliares para evitar revisões globais da ár-

vore. Caso seja necessário, a abordagem será restrita a um conjunto de atributos e operações de menor complexidade.

7.0.4 Implementação e reprodutibilidade

A implementação do algoritmo pode revelar problemas não observados na formulação matemática, como erros de representação da árvore, tratamento de casos degenerados e inconsistências na manutenção das componentes afetadas.

Como contingência, a implementação será modularizada e acompanhada de testes unitários para subrotinas específicas. Além disso, serão definidos critérios de reprodutibilidade, métricas padronizadas e conjuntos de dados de referência para avaliar a consistência da solução.

7.0.5 Validação experimental

Há risco de que os ganhos observados em termos de desempenho computacional não sejam suficientemente relevantes em alguns cenários de aplicação, reduzindo o impacto prático da proposta.

Como contingência, a avaliação experimental será planejada antes da implementação, com métricas bem definidas para tempo de execução, uso de memória e número de operações locais. A comparação com a reconstrução completa da *max tree* será central para demonstrar o benefício real do operador.

7.0.6 Cronograma e escopo da pesquisa

Devido à combinação de teoria, algoritmo, implementação e experimentação, o projeto pode sofrer desvio de cronograma ou expansão de escopo em uma de suas etapas.

Como contingência, o cronograma será revisado periodicamente, e os objetivos serão organizados em marcos intermediários. Caso uma extensão teórica ou experimental se mostre mais complexa do que o previsto, a pesquisa será priorizada para os elementos essenciais da contribuição: formalização, eficiência e validação do operador de *split*.

7.0.7 Dependência da literatura

A literatura sobre atualização dinâmica de *component trees* e sobre o operador dual de *split* ainda é relativamente escassa, o que pode dificultar a comparação direta com o estado da arte.

Como contingência, a revisão bibliográfica será contínua e integrada ao desenvolvimento do trabalho. Sempre que necessário, novas referências serão incorporadas ao arcabouço teórico para situar a contribuição no contexto da morfologia matemática e das estruturas hierárquicas de imagens.

Referências

[1] Wonder A.L. Alves, Charles F. Gobber, Dennis J. Silva, Alexandre Morimitsu, Ronaldo F. Hashimoto, and Beatriz Marcotegui. Image segmentation based on ultimate levelings: From attribute filters to machine learning strategies. *Pattern Recognition Letters*, 133:264–271, May 2020.

[2] D. L. Farfan Cabrera, N. Gogin, D. Morland, B. Naegel, D. Papathanassiou, and N. Passat. Segmentation of Axillary and Supraclavicular Tumoral Lymph Nodes in PET/CT: A Hybrid CNN/Component-Tree Approach. In *2020 25th International Conference on Pattern Recognition (ICPR)*, pages 6672–6679, Milan, Italy, January 2021. IEEE.

[3] Gregory M. Hunter and Kenneth Steiglitz. Operations on Images Using Quad Trees. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2):145–153, April 1979.

[4] Wonder Alexandre Luz Alves, Nicolas Passat, Dennis J. Silva, Alexandre Morimitsu, and Ronaldo F. Hashimoto. Component Tree: Update Rather than Rebuild. *Journal of Mathematical Imaging and Vision*, 68(5):61, August 2026.

[5] Cyril Meyer, Étienne Baudrier, Patrick Schultz, and Benoît Naegel. Combining Max-Tree and CNN for Segmentation of Cellular FIB-SEM Images. In Bertrand Kerautret, Miguel Colom, Adrien Krähenbühl, Daniel Lopresti, Pascal Monasse, and Benjamin Perret, editors, *Reproducible Research in Pattern Recognition*, volume 14068, pages 77–90. Springer Nature Switzerland, Cham, 2023.

[6] L. Najman and M. Couprie. Building the Component Tree in Quasi-Linear Time. *IEEE Transactions on Image Processing*, 15(11):3531–3539, November 2006.

[7] L. Najman and M. Schmitt. Geodesic saliency of watershed contours and hierarchical segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 18(12):1163–1173, December 1996.

[8] Laurent Najman and Jean Cousty. A graph-based mathematical morphology reader. *Pattern Recognition Letters*, 47:3–17, October 2014.

[9] Benjamin Perret. Inf-structuring Functions: A Unifying Theory of Connections and Connected Operators. *Journal of Mathematical Imaging and Vision*, 51(1):171–194, January 2015.

[10] Azriel Rosenfeld. Connectivity in Digital Pictures. *Journal of the ACM*, 17(1):146–160, January 1970.

[11] P. Salembier and L. Garrido. Binary partition tree as an efficient representation for image processing, segmentation, and information retrieval. *IEEE Transactions on Image Processing*, 9(4):561–576, April 2000.

[12] P. Salembier, A. Oliveras, and L. Garrido. Antiextensive connected operators for image and sequence processing. *IEEE Transactions on Image Processing*, 7(4):555–570, April 1998.

[13] P. Salembier and J. Serra. Flat zones filtering, connected operators, and filters by reconstruction. *IEEE Transactions on Image Processing*, 4(8):1153–1160, August 1995.

[14] Philippe Salembier and Michael Wilkinson. Connected operators. *IEEE Signal Processing Magazine*, 26(6):136–157, November 2009.

[15] S. Tanimoto and T. Pavlidis. A hierarchical data structure for picture processing. *Computer Graphics and Image Processing*, 4(2):104–119, June 1975.